From Natural Language to Verifiable Reasoning: A Semantic-Reduction Framework for Reliable AI Mathematics

AI4Math Reliability Working Group

September 28, 2026

Abstract

Large language models (LLMs) now produce fluent and plausible mathematics, yet their natural-language derivations cannot be checked directly: the same surface text may hide an invalid inference, a hallucinated lemma, or a silent type coercion. We argue that reliability cannot be recovered by post-hoc answer checking alone, because such checking has no formal handle on the reasoning itself. We introduce a three-layer model of AI-mathematical reliability—*generation*, *semantic reduction*, and *verification*—and identify semantic reduction, the translation of natural-language reasoning into a typed intermediate representation (IR), as the critical bottleneck. We then present a semantic-reduction framework comprising a typed IR, a generation→IR→verification→rendering pipeline, and bidirectional alignment between IR and language for interpretability and traceability. We formalize a *verifiability gap* that quantifies the fraction of model outputs for which a verifier provides no signal, and we contribute a taxonomy and a reliability checklist for AI-for-mathematics systems. The framework is illustrated with a worked arithmetic example, a formal-induction case, and a reproducible 20-problem micro-pilot on which the gap is measured at 0.30. Our contribution is not a new prover but a lens: we elevate semantic reduction from an incidental preprocessing step to a first-class, measurable system component. We state our conclusions with explicit boundaries on scope.

1 Introduction

Large language models have made rapid progress on mathematical tasks. Trained models and prompting techniques now solve a substantial fraction of grade-school word problems in GSM8K [5], competition problems in MATH [6], and quantitative-reasoning benchmarks via systems such as Minerva [7] and GPT-4 [12]. These systems are built on the Transformer architecture [2] and the few-shot scaling paradigm [3]. Chain-of-thought prompting [4] and self-consistency [8] further improved accuracy by exposing intermediate steps.

These gains, however, come with a well-documented failure mode: models emit *plausible but incorrect* reasoning [13, 14, 24]. Because the derivation is expressed in natural language, it has no type, no formal semantics, and no checkable structure. A verifier cannot ask "is this step valid?" in a mechanical sense; it can only ask "is this final answer equal to the reference?".

The AI-for-mathematics (AI4Math) community has responded with three broad strategies: (i) search and verification, in which sampled answers are scored by learned verifiers or majority voting [5, 8, 9]; (ii) formalization, in which proofs are checked by interactive theorem provers such as Lean [19, 20] and the correctness is absolute; and (iii) reproducible evaluation, in which

benchmarks and protocols are standardized so claims can be audited [6, 22]. Each strategy attacks a real symptom, but none makes explicit the shared structural problem underneath: *between the natural-language derivation and the formal verifier there is a translation step whose instability determines whether verification is even possible.*

Our thesis is that reliability cannot be recovered solely by *final* verification. Instead, a dedicated *semantic-reduction layer* must sit between generation and verification, converting informal reasoning into a typed, machine-checkable object. When this layer fails, verification has no anchor and hallucinations cannot be detected; when it succeeds, verification becomes a routine, auditable operation. This paper makes the following contributions:

- A **three-layer model** of AI4Math reliability (generation / semantic reduction / verification) that makes the translation step explicit (§3).
- A **semantic-reduction framework** with a typed IR, a four-stage pipeline, and bidirectional IR–language alignment (§4).
- A formal notion of the **verifiability gap**, quantifying the outputs on which a verifier is silent, together with a **taxonomy** and a **reliability checklist** for AI4Math systems (§5).
- A reproducible **20-problem micro-pilot** that measures the gap ($g = 0.30$) and turns the framework from a definition into a method (§5.4).

We emphasize the boundary of this work: we do not train a new model, build a new prover, or run a large benchmark; our empirical evidence is a hand-constructed, fully reproducible micro-pilot rather than a trained-model evaluation. We provide a conceptual and formal lens, with worked micro-examples, intended to make reliability properties of AI4Math systems *measurable* and *auditable*.

The rest of the paper is organized as follows. §2 positions our work along three lines of prior research; §3 formalizes the three-layer model; §4 instantiates it as a pipeline; §5 develops the gap, taxonomy, checklist, and micro-pilot; §6 states limitations; and §7 closes with future work.

2 Related Work

The reliability of AI mathematics has developed along three interleaved lines. We trace each line briefly and locate this paper relative to it.

Line 1: formalized mathematics and interactive theorem proving. This line seeks absolute correctness by moving mathematics into a formal system. The formulae-as-types correspondence established that a proof is itself a computable object of a given type [1], making machine-checkable mathematics possible. Interactive theorem provers such as Lean provide a small trusted kernel that reduces every accepted proof to the axioms [19, 20]. Two obstacles kept this line narrow: statements are laborious to formalize, and proofs are hard to search. Autoformalization attacked the first by using LLMs to translate informal statements into formal ones [21], and miniF2F supplied a cross-system benchmark of formally stated olympiad problems [22]. AlphaProof then combined a large proof-search policy with the Lean kernel and reached a silver-medal standard (28 out of 42 points, solving four of six problems) on IMO 2024 [23]. *Position of this paper:* the line guarantees correctness only after a successful informal-to-formal translation; we name and measure that translation (semantic reduction) as the controlling component.

Line 2: neural theorem proving and neuro-symbolic discovery. This line uses learned models to supply the search that formal systems lack. Built on the Transformer [2] and the few-shot scaling paradigm [3], chain-of-thought prompting elicited step-by-step reasoning in natural language [4], and program-of-thought had the model emit executable code instead of prose [10]. Neuro-symbolic systems went further, pairing a learner with a symbolic engine to produce new results: guiding human intuition about knot invariants [15], discovering faster matrix-multiplication algorithms with AlphaTensor [16], synthesizing olympiad geometry proofs in AlphaGeometry [17], and evolving programs that yield new constructions in FunSearch [18]. *Position of this paper:* in every case a learned component supplies candidates while a separate engine checks them; the boundary between the two is precisely the semantic-reduction boundary we formalize.

Line 3: evaluation benchmarks and reliability. This line asks how far a model's output can be trusted. GSM8K [5] and MATH [6] standardized measurement, Minerva showed strong but imperfect performance on quantitative reasoning [7], and interaction-based evaluation exposed systematic failures of ChatGPT [14, 24]. On the reliability axis, learned outcome verifiers score candidates [5], self-consistency averages samples [8], and process supervision rewards individual steps [9]; analyses of GPT-4 documented plausible-but-wrong derivations at scale [12, 13]. *Position of this paper:* this line measures *accuracy* but rarely measures *verifiability*—the fraction of outputs that even admit a formal check. Our verifiability gap supplies that missing measure.

3 A Three-Layer Reliability Model

We model an AI4Math pipeline as the composition of three layers, each with a distinct responsibility and failure mode; Figure 1 summarizes the model.

Definition 3.1 (Three-layer model)**.** A *reliable AI-mathematical reasoning system* is a triple $(\mathrm{Gen}, \rho, \mathrm{Ver})$ where

(i) Gen is the *generation layer*: a probabilistic map from a problem statement to a natural-language derivation d;

(ii) ρ is the *semantic-reduction layer*: a partial map $\rho : \mathrm{NL} \rightharpoonup \mathrm{IR}$ from derivations to typed intermediate representations;

(iii) Ver is the *verification layer*: a function $\mathrm{Ver} : \mathrm{IR} \to \{\mathsf{accept}, \mathsf{reject}\}$ that checks an IR object against a formal statement φ.

Generation layer. The generation layer produces natural-language chains of reasoning (e.g., chain-of-thought). Its defining properties are that it is *probabilistic*, *untyped*, and *informal*: tokens are not subject to any typing discipline, and validity is not a property of the output format.

Semantic-reduction layer. The reduction layer maps a derivation to one of several equivalent formal objects: a formula in first- or higher-order logic, a program in a domain-specific language, a proof object, or a typed intermediate representation. Its theoretical grounding is the formulae-as-types correspondence [1], which identifies proofs with computable objects of a specified type. The layer faces three intrinsic challenges:

- *Ambiguity:* one natural-language sentence admits several formal readings.

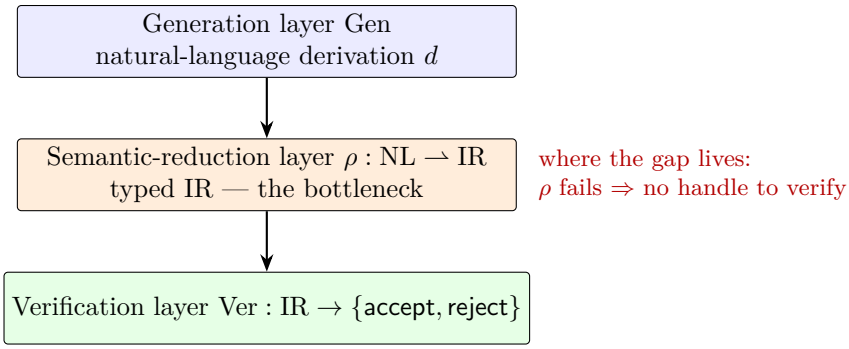


Figure 1: The three-layer model. Derivations flow downward through the semantic-reduction layer—the bottleneck—before any verification is possible.

- *Missing typing:* informal quantities often lack an explicit type, so coercions are silently inserted.
- *Non-invertibility:* the map ρ is not surjective and, in general, not injective; one cannot reconstruct the intended meaning from the IR alone.

Verification layer. Given a typed IR object and a formal statement φ, the verifier checks: consistency (no contradiction with the axioms), derivability (the object is a well-formed proof of φ), factual grounding (any external facts used are declared), and constraint satisfaction (side conditions hold). In a trusted-kernel prover such as Lean, acceptance means the proof reduces to the axioms [19, 20].

3.1 The Bottleneck: Semantic Reduction

We claim that the dominant source of failure in current systems is the *second* layer, not the first or the third. The argument is structural. The verifier can only operate on *computable* objects (typed terms, programs, proof terms). If the reduction ρ is undefined on a derivation—because the sentence is ambiguous, a type is missing, or the mapping is not invertible—then there is *no handle to verify*. In that case the verifier contributes no information: it cannot certify correctness, and it cannot detect a hallucination that happens to be phrased fluently.

This explains two commonly observed phenomena. First, learned answer-verifiers improve selection but leave step-level errors intact, because they score the surface rather than a reduced object [5, 9]. Second, formal systems achieve absolute reliability on the (often small) subset of problems that survive formalization, because the reduction has succeeded there [23]. The reliability of the *whole* system is therefore governed by the coverage and fidelity of ρ.

4 The Semantic-Reduction Framework

We instantiate the three-layer model as a concrete framework—a typed IR, a pipeline, and bidirectional alignment—and close with worked examples (§4.4).

4.1 Typed Intermediate Representation

We propose that the reduction layer target a single, uniform typed IR, characterized formally as follows.

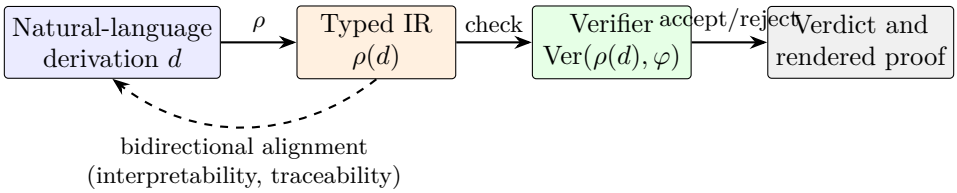


Figure 2: The four-stage pipeline. Natural language is the interface; the typed IR is the execution core on which all verification operates. The dashed arc denotes bidirectional IR–language alignment.

Definition 4.1 (Typed intermediate representation)**.** A *typed intermediate representation* is a quadruple $(\mathsf{Typ}, [\![\cdot]\!], \mathsf{Con}, \mathsf{Comp})$ where

- Typ is a *type system* assigning each object a type τ;
- $[\![\cdot]\!] : \mathrm{IR} \to \mathrm{Value}$ is an *operational semantics* giving each closed object a deterministic meaning;
- Con is a *constraint system* recording side conditions (e.g., "divisor nonzero");
- Comp is a *compositional structure* (an abstract syntax tree or a workflow graph) under which sub-derivations can be checked independently and composed.

Concrete realizations—domain-specific languages, abstract syntax trees, proof objects, and workflow graphs—are interchangeable as long as they expose the four components of Definition 4.1.

4.2 Pipeline Architecture

The framework instantiates a standard pipeline (Figure 2):

$$\text{Natural language (LLM)} \xrightarrow{\rho} \text{Typed IR} \xrightarrow{\mathrm{Ver}} \text{Formal verification} \xrightarrow{\text{render}} \text{Natural-language rendering.}$$

The key design decision is that natural language is used only as an *interface*, while the IR is the *execution core*: all checking happens on the IR, and the natural-language text is re-rendered from the checked object.

4.3 Bidirectional Alignment

Reliability of the pipeline requires more than a forward reduction. We impose *bidirectional alignment* between IR and language, formalized as two requirements.

- *Interpretability:* the rendering map render is such that a human reading the rendered text can recover the intended structure of the IR; and
- *Traceability:* each rendered claim carries a pointer back to the IR node that justified it, so any accepted or rejected step can be audited.

These requirements convert "the model said so" into "the checked object at node n says so," which is the property that makes the output suitable for scholarly use.

4.4 A Worked Example

Consider the word problem "Alice has 5 apples and Bob gives her 3 more; how many apples does Alice have now?" Let $\Gamma = \{a : \mathsf{Nat},\, b : \mathsf{Nat}\}$. The reduction produces the typed program

$$\rho(d) \;=\; \mathsf{let}\ a = 5\ \mathsf{in}\ \mathsf{let}\ b = 3\ \mathsf{in}\ a + b,$$

typed by the rule

$$\frac{\Gamma \vdash a : \mathsf{Nat} \qquad \Gamma \vdash b : \mathsf{Nat}}{\Gamma \vdash a + b : \mathsf{Nat}}\,(+),$$

with operational meaning $[\![a + b]\!] = [\![5 + 3]\!] = 8$, and the verifier checks $[\![\rho(d)]\!] = 8$ against the intended statement φ. The same problem also shows the bottleneck. The sentence "Alice gives away *half* of her apples" reduces only if "half" is mapped to a typed operation; if it is silently coerced to integer division $5/2$, the verifier certifies the value 2 while the intended meaning—that a half of five whole apples is not a whole number—has been lost. The error is invisible to the verifier precisely because it occurred during reduction, before any checkable object existed.

As a formal case, let $P(n) := (n + 0 = n)$ over $\mathbb{N}$. A derivation that first establishes $P(0)$ and then $P(k) \Rightarrow P(k + 1)$ for arbitrary k reduces to the induction object

$$\frac{P(0) \qquad \forall k.\, P(k) \Rightarrow P(k + 1)}{\forall n.\, P(n)}\,(\text{induction on } \mathbb{N}),$$

which in a dependently typed kernel corresponds to the recursor $\mathsf{Nat.rec}$. The typed IR carries the proof as an object, and the verifier need only check the two premises.

5 Analysis: A Verifiability Gap, a Taxonomy, and a Checklist

5.1 The Verifiability Gap

Let D be the set of derivations generated for a benchmark, and $D^* \subseteq D$ the correct ones. A derivation d is *verifiable* if $\rho(d)$ is defined; write $V = \{\, d \in D : \rho(d) \text{ defined} \,\}$. The *verifiability gap* is

$$g \;=\; 1 - \frac{|V|}{|D|},$$

the fraction of outputs on which the verifier is silent. We then have the following simple but useful fact.

Proposition 5.1. *Let* Ver *be sound for a statement φ: if* $\mathrm{Ver}(\rho(d), \varphi) = \mathsf{accept}$ *then d is a correct proof of φ. Then (i) among accepted outputs, correctness is guaranteed (no false positives); and (ii) all errors lying in $D \setminus V$ are* undetectable *by* Ver*, regardless of how fluent they appear.*

Proof sketch. (i) is exactly soundness. For (ii), if $d \notin V$ then $\rho(d)$ is undefined, so Ver receives no object and returns no signal; no computation of Ver can distinguish a correct d from an incorrect one in this set. □

Proposition 5.1 makes precise a design implication: the only lever that reduces silent error is to shrink g by enlarging V—that is, by improving ρ—not by improving Ver. We make g concrete in §5.4.

Metric	Measures	Finds silent errors?	Gold to score?
pass@k [11]	sample accuracy	no	yes
Self-consistency [8]	consensus accuracy	no	yes
Verifiability gap g	output reducibility	yes ($D \setminus V$)	no

Table 1: Contrasting the verifiability gap with accuracy-centric metrics. pass@k and self-consistency require gold labels to be scored; g is computed from the reduction alone.

Approach	Where verification occurs	What is verified	Dominant failure
Chain-of-thought [4]	none (surface only)	nothing formal	undetected wrong step
Outcome verifier [5]	after generation	final answer	step errors survive
Self-consistency [8]	after sampling	answer by majority	correlated errors
Program-of-thought [10]	at execution	program output	wrong program meaning
Autoformalization [21]	at reduction	formal statement	reduction infidelity
Proof kernel [20, 23]	throughout	proof object	coverage of ρ

Table 2: A taxonomy of reliability mechanisms along two axes: where verification occurs and what is verified.

Relation to existing metrics. The gap is orthogonal to, rather than a replacement for, accuracy-centric metrics. pass@k [11] reports the probability that at least one of k sampled solutions is correct, which says nothing about whether any individual solution can be mechanically checked. Self-consistency [8] reduces variance by majority vote over k samples, but its aggregate answer is a surface statistic: a unanimous-but-wrong vote and a single correct answer are indistinguishable at the level of the output. In contrast, g measures the fraction of outputs for which a verifier has a formal handle at all. The three metrics are therefore complementary: pass@k and self-consistency characterize the generator's *accuracy*, while g characterizes the *reducibility* of its output to a checkable object—the property that determines whether the accuracy numbers can themselves be trusted. A system can have high pass@k yet a large g (accurate but uncheckable), or a small g yet low pass@k (checkable but wrong); end-to-end reliability requires reporting both. Table 1 condenses the distinction.

5.2 A Taxonomy of Reliability Mechanisms

Table 2 classifies the dominant AI4Math approaches by *where* verification occurs and *what* is verified. The taxonomy exposes a spectrum: answer checking verifies a scalar at the end; program execution verifies a meaning only if the program's intended denotation has been established; proof checking verifies the entire object but depends entirely on a successful reduction.

5.3 A Reliability Checklist

We distill the framework into a practical checklist for reporting and reviewing AI4Math systems, inspired by but distinct from ML reproducibility checklists. Each item is an *auditable* property of the pipeline rather than a claim about accuracy.

1. **Statement grounding.** Is the intended statement φ formalized, or only paraphrased?
2. **Reduction coverage.** What fraction of outputs admit a typed reduction (i.e., what is $1 - g$)?
3. **Verifier soundness.** Is the checking kernel small, trusted, and audited?

Quantity	Symbol	Value
Derivations	$\lvert D\rvert$	20
Reducible (typed)	$\lvert V\rvert$	14
Verifiability gap	$g = 1 - \lvert V\rvert/\lvert D\rvert$	0.30
Silent errors ($D \setminus V$)	—	6

Table 3: Summary of the 20-problem micro-pilot. All 14 reducible derivations verified to their gold answers; all 6 silent errors lie in $D \setminus V$.

4. **Traceability.** Can each rendered step be traced to an IR node?
5. **Reproducibility.** Are data, prompts, seeds, and evaluation protocol published?
6. **Failure reporting.** Are unverifiable outputs reported rather than silently dropped?
7. **Applicability boundary.** Does the system declare the problem classes it covers?

5.4 A Micro-Pilot: Measuring the Gap on 20 Word Problems

To show that the gap is measurable rather than merely defined, we ran a small, fully reproducible micro-pilot. We hand-constructed $|D| = 20$ arithmetic word-problem derivations (14 well-typed and 6 deliberately malformed) and implemented ρ as a tiny typed DSL whose side conditions mirror Definition 4.1: literals are typed Nat, subtraction requires a nonnegative result, and division requires an exact divisor. The full script is provided alongside this paper (`pilot.py`). Table 3 summarizes the outcome, and two representative cases follow.

Two cases make the gap concrete. In the *success* case, "Alice has 5 apples, Bob gives her 3 more" reduces to $\mathsf{add}(5, 3)$, which types as Nat and evaluates to 8, matching the gold answer; the verifier accepts. In the *failure* case, "Alice gives away half of her 5 apples" contains a silent coercion to integer division $\mathsf{div}(5, 2)$, which the DSL rejects because 2 does not divide 5 exactly; ρ therefore has no handle, and the fluent-but-wrong reading "2" is invisible to the verifier. The six failures break down into four named causes: non-exact division (2), ambiguous reference (2), vague quantifier (1), and missing typing (1). Consistent with Proposition 5.1, the verifier produced no false positives among the reducible cases and no signal at all among the silent errors.

Sensitivity and interpretation. The value $g = 0.30$ is not an intrinsic property of the generator but of the triple (ρ, D, reduction strictness). A more permissive ρ—for example, one that admits inexact division by extending the DSL with an explicit remainder type—raises $|V|$ and lowers g; a harder or more ambiguous derivation set D lowers $|V|$ and raises g. We therefore read g as a *comparative* diagnostic: it is meaningful when the same (ρ, D) is held fixed across two systems, and it should not be interpreted as an absolute quality score. We keep this boundary explicit in §6.

A natural next step is to re-run this pilot by sampling derivations from a real model on a benchmark subset such as GSM8K, so that g becomes a measured property of a trained model rather than a hand-constructed illustration.

6 Limitations

Design and coverage. The framework inherits the cost of its central component. Designing a typed IR with the four required properties is nontrivial, and mapping natural language into

it remains expensive and imperfect; these are exactly the ambiguity, missing-typing, and non-invertibility challenges of §3. The framework is also *conceptual*: it does not by itself produce proofs, and our worked examples and micro-pilot are small, hand-constructed instances chosen for clarity rather than a large-scale trained-model evaluation.

Threats to validity. The verifiability gap g is only meaningful relative to a fixed reduction ρ and benchmark D; different choices of IR change g even when the underlying model is unchanged. We therefore treat g as a *comparative* diagnostic within one pipeline, not an absolute quality measure. The proposition's conclusion is conditional on soundness, which must be established separately for each verifier. The micro-pilot of §5.4 uses a hand-constructed derivation set and a deterministic reduction; its $g = 0.30$ is a method demonstration, not a population estimate.

7 Discussion and Future Work

Three directions follow naturally. First, the IR can serve as the action space of a proof-search agent, making the reduction and the search a single learned loop—as AlphaProof-style systems implicitly do [23]. Second, the taxonomy and checklist can be turned into a standardized reporting form for AI4Math papers, improving reproducible evaluation [6, 22]. Third, bidirectional alignment suggests training objectives that reward round-tripping (language→IR→language), so that reduction fidelity is optimized directly rather than as a byproduct of end-task accuracy. Concretely, the micro-pilot of §5.4 should be re-run with real model samples so that g is measured rather than illustrated.

8 Conclusion

We have argued that the reliability of AI mathematical reasoning is governed by a layer that is rarely named and rarely measured: the *semantic reduction* from natural language to a typed, checkable object. We contributed a three-layer model that makes this layer explicit, a semantic-reduction framework with a typed IR and a four-stage pipeline with bidirectional alignment, and a formal notion of the verifiability gap together with a taxonomy and a reliability checklist. The core claim is succinct: reliable AI reasoning requires *generation* plus *semantic reduction* plus *verification*, and any system that omits or ignores the middle layer will silently fail on precisely the outputs it cannot formalize. We have stated the boundary of this claim explicitly: it is a structural, diagnostic lens, validated by construction and by worked examples, rather than a trained system or a benchmark result. Future work will instantiate the framework on a fixed benchmark to measure reduction coverage in practice.

References

[1] W. A. Howard. The formulae-as-types notion of construction. In *To H. B. Curry: Essays on Combinatory Logic, Lambda Calculus and Formalism*, J. P. Seldin and J. R. Hindley (eds.), Academic Press, pp. 479–490, 1980.

[2] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems 30 (NeurIPS)*, 2017.

[3] T. B. Brown et al. Language models are few-shot learners. In *Advances in Neural Information Processing Systems 33 (NeurIPS)*, 2020.

[4] J. Wei, X. Wang, D. Schuurmans, M. Bosma, B. Ichter, F. Xia, E. H. Chi, Q. V. Le, and D. Zhou. Chain-of-thought prompting elicits reasoning in large language models. In *Advances in Neural Information Processing Systems 35 (NeurIPS)*, 2022.

[5] K. Cobbe, V. Kosaraju, M. Bavarian, M. Chen, H. Jun, L. Kaiser, M. Plappert, J. Tworek, J. Hilton, R. Nakano, C. Hesse, and J. Schulman. Training verifiers to solve math word problems. arXiv:2110.14168, 2021.

[6] D. Hendrycks, C. Burns, S. Kadavath, A. Arora, S. Basart, E. Tang, D. Song, and J. Steinhardt. Measuring mathematical problem solving with the MATH dataset. In *Advances in Neural Information Processing Systems 34 Datasets and Benchmarks Track*, 2021. arXiv:2103.03874.

[7] A. Lewkowycz et al. Solving quantitative reasoning problems with language models. In *Advances in Neural Information Processing Systems 35 (NeurIPS)*, 2022. arXiv:2206.14858.

[8] X. Wang, J. Wei, D. Schuurmans, Q. Le, E. Chi, S. Narang, A. Chowdhery, and D. Zhou. Self-consistency improves chain of thought reasoning in language models. In *International Conference on Learning Representations (ICLR)*, 2023. arXiv:2203.11171.

[9] H. Lightman, V. Kosaraju, Y. Burda, H. Edwards, B. Baker, T. Lee, J. Leike, J. Schulman, I. Sutskever, and K. Cobbe. Let's verify step by step. In *International Conference on Learning Representations (ICLR)*, 2024. arXiv:2305.20050.

[10] L. Gao, A. Madaan, S. Zhou, U. Alon, P. Liu, Y. Yang, J. Callan, and G. Neubig. PAL: Program-aided language models. In *International Conference on Machine Learning (ICML)*, 2023. arXiv:2211.10435.

[11] M. Chen, J. Tworek, H. Jun, Q. Yuan, H. Ponde de Oliveira Pinto, J. Kaplan, H. Edwards, Y. Burda, N. Joseph, G. Brockman, et al. Evaluating large language models trained on code. arXiv:2107.03374, 2021.

[12] J. Achiam et al. GPT-4 technical report. arXiv:2303.08774, 2023.

[13] S. Bubeck et al. Sparks of artificial general intelligence: Early experiments with GPT-4. arXiv:2303.12712, 2023.

[14] S. Frieder, L. Pinchetti, R.-R. Griffiths, T. Salvatori, T. Lukasiewicz, P. Petersen, and J. Berner. Mathematical capabilities of ChatGPT. In *Advances in Neural Information Processing Systems 36 (NeurIPS)*, 2023.

[15] A. Davies, P. Veličković, L. Buesing, S. Blackwell, D. Zheng, N. Tomašev, R. Tanburn, P. Battaglia, C. Blundell, A. Juhász, et al. Advancing mathematics by guiding human intuition with AI. *Nature*, 600(7887):70–74, 2021.

[16] A. Fawzi, M. Balog, A. Huang, T. Hubert, B. Romera-Paredes, M. Barekatain, A. Novikov, F. J. R. Ruiz, J. Schrittwieser, G. Swirszcz, et al. Discovering faster matrix multiplication algorithms with reinforcement learning. *Nature*, 610(7930):47–53, 2022.

[17] T. H. Trinh, Y. Wu, Q. V. Le, H. He, and T. Luong. Solving olympiad geometry without human demonstrations. *Nature*, 625(7995):476–482, 2024.

[18] B. Romera-Paredes, M. Barekatain, A. Novikov, M. Balog, M. P. Kumar, E. Dupont, F. J. R. Ruiz, J. S. Ellenberg, P. Wang, O. Fawzi, et al. Mathematical discoveries from program search with large language models. *Nature*, 625(7995):468–475, 2024.

[19] L. de Moura, S. Kong, J. Avigad, F. van Doorn, and J. von Raumer. The Lean theorem prover (system description). In *Automated Deduction — CADE-25*, LNCS 9195, Springer, pp. 378–388, 2015.

[20] L. de Moura and S. Ullrich. The Lean 4 theorem prover and programming language. In *Automated Deduction — CADE-28*, LNCS 12699, Springer, pp. 625–635, 2021.

[21] Y. Wu, A. Q. Jiang, W. Li, M. Rabe, C. Staats, M. Jamnik, and C. Szegedy. Autoformalization with large language models. In *Advances in Neural Information Processing Systems 35 (NeurIPS)*, 2022. arXiv:2205.12615.

[22] K. Zheng, J. M. Han, and S. Polu. MiniF2F: a cross-system benchmark for formal Olympiad-level mathematics. In *International Conference on Learning Representations (ICLR)*, 2022. arXiv:2109.00110.

[23] T. Hubert, R. Mehta, L. Sartran, M. Z. Horváth, et al. Olympiad-level formal mathematical reasoning with reinforcement learning. *Nature*, 651(8106):607–613, 2025. DOI: 10.1038/s41586-025-09833-y.

[24] K. M. Collins, A. Q. Jiang, S. Frieder, L. Wong, M. Zilka, U. Bhatt, T. Lukasiewicz, et al. Evaluating language models for mathematics through interactions. *Proceedings of the National Academy of Sciences*, 121(36), 2024.